

Qiskit Developer Certification

Revision Notes

qiskit.circuit — Quantum Circuit Model

Source: IBM Quantum Documentation | quantum.cloud.ibm.com/docs/en/api/qiskit/circuit

1 Introduction to qiskit.circuit

The `qiskit.circuit` module is the core of Qiskit. It defines the quantum circuit — the fundamental unit of execution on a Quantum Processing Unit (QPU).

A circuit is a computational routine that runs one shot at a time on a QPU. It acts on quantum data (qubits) using unitary operations (gates), measurements, and resets. It can also include real-time classical computation and control-flow constructs executed by the QPU's controllers.

1.1 Two Regimes of Circuits

Regime	Definition	Qubit Type	How to Obtain
Abstract Circuit	Defined using virtual qubits and arbitrary high-level operations	Virtual qubits — assume all gates and connections are valid	Directly authored by the programmer
Physical Circuit (ISA Circuit)	Defined using hardware qubits and only the QPU's native operations	Hardware qubits — constrained by backend coupling and ISA	Run <code>transpile()</code> on an abstract circuit

Key method: Use `QuantumCircuit.ensure_physical()` to mark a circuit as physical without transpiling. This does NOT enforce ISA constraints — it only updates metadata.

2 Key Concepts & Terminology

Term	Definition
Qubit	The basic unit of quantum information

Clbit	Qiskit-specific abbreviation for a single classical bit of data
Gate	A unitary operation on one or more qubits
Measurement	Extracts one classical bit from a qubit state — irreversible, destroys entanglement
Ancilla qubit	An extra qubit used to help implement operations; its final state does not matter
Virtual qubit	An abstract qubit in an abstract circuit — all gates and connections assumed valid
Hardware qubit	A qubit on a specific QPU — subject to coupling and ISA constraints
Physical qubit	A literally controllable two-level quantum system (e.g., a transmon or trapped ion)
Logical qubit	Several physical qubits grouped together for quantum error correction
QPU	Quantum Processing Unit — the hardware that runs quantum operations
ISA	Instruction Set Architecture — defines which gates are available on which hardware qubits
Unitary operation	A reversible quantum operation — all gates are unitary by definition
Real-time classical computation	Classical computation inside a single shot of a circuit, whose results can affect later gates
Abstract circuit	A circuit in terms of virtual qubits and mathematical operations — no hardware constraints
Physical circuit	A circuit in terms of hardware qubits and native operations only

3 Bits and Registers

Bits and registers are the raw data containers in Qiskit circuits.

3.1 Bit Classes

Class	Base Class	Description
Bit	(object)	Abstract base — cannot be instantiated directly. Only for type-checking.
Qubit	Bit	A single quantum bit. Instantiate with <code>Qubit()</code> or via <code>QuantumRegister</code> .
Clbit	Bit	A single classical bit. Instantiate with <code>Clbit()</code> or via <code>ClassicalRegister</code> .

AncillaQubit	Qubit	A qubit flagged as ancilla. Rarely used in practice.
--------------	-------	--

Equality rule: Two bits compare equal if they are the same Python object, OR if they were created by a register of the same name and size at the same index.

3.2 Register Classes

Class	Base Class	Constructor	Contains
Register	(object)	Cannot instantiate directly — base only	—
QuantumRegister	Register	QuantumRegister(size, name) or QuantumRegister(name, bits=[...])	Qubit objects
ClassicalRegister	Register	ClassicalRegister(size, name)	Clbit objects
AncillaRegister	QuantumRegister	AncillaRegister(size, name)	AncillaQubit objects

```
from qiskit.circuit import QuantumRegister, ClassicalRegister, QuantumCircuit

qr = QuantumRegister(3, 'q')    # 3 qubits named 'q'
cr = ClassicalRegister(3, 'c')  # 3 classical bits named 'c'
circ = QuantumCircuit(qr, cr)
```

Important: ClassicalRegister can be implicitly cast to an unsigned integer for use in control-flow condition comparisons. Regular classical bits cannot.

4 Circuit Instruction Anatomy

A QuantumCircuit internally stores a sequence of CircuitInstructions. Each one pairs an operation with its qubit and clbit operands.

4.1 CircuitInstruction

Field	Type	Description
operation	Operation / Instruction / Gate	What to do — the gate or instruction
qubits	tuple of Qubit	Which qubits to act on

clbits	tuple of Clbit	Which classical bits to act on (if any)
--------	----------------	---

Critical note: `CircuitInstruction` is implemented in Rust. Mutating a `CircuitInstruction` object does NOT update the parent circuit. To modify a circuit's instructions, use the transpiler's `DAGCircuit` interface.

4.2 InstructionSet

Most `QuantumCircuit` methods (`append()`, `h()`, `cx()`, etc.) return an `InstructionSet` — a handle to one or more `CircuitInstructions` that were just added. It is rarely needed in modern Qiskit and its mutation methods are discouraged.

5 Class Hierarchy: Operation → Instruction → Gate

5.1 Operation (most abstract)

- Minimal interface — only `name`, `num_qubits`, `num_clbits`
- Allows arbitrary mathematical objects (e.g., `quantum_info.Operator`) to be added to abstract circuits
- Cannot be used in physical circuits

5.2 Instruction (concrete, hardware-runnable)

- Subclass of `Operation` — represents something a QPU could conceivably run natively
- Need NOT be unitary — e.g., `Measure` and `Reset` are `Instructions` but not `Gates`
- Has a definition attribute: a `QuantumCircuit` that decomposes the instruction into simpler ops
- `definition=None` means the instruction is opaque to optimizers
- Parameters stored in `params` — should be real numbers (angles) wherever possible

```
# Instruction constructor signature
Instruction(name: str, num_qubits: int, num_clbits: int, params: list)
```

5.3 Gate (unitary instruction)

- Subclass of `Instruction` — specifically unitary
- Additional methods: `to_matrix()` and `control()`
- Cannot act on classical resources (no `clbits`)
- Can have multiple decompositions registered in an `EquivalenceLibrary`

5.4 ControlledGate

- Subclass of Gate — adds control-qubit semantics
- Base of many standard-library gates: CXGate, CCXGate, CZGate, etc.

5.5 AnnotatedOperation

- A subclass of Operation with applied abstract modifiers — used for lazy transformation

Modifier	Effect
InverseModifier	Marks the operation as the inverse (adjoint) — applied lazily
ControlModifier	Adds control qubits to the operation
PowerModifier	Raises the operation to a given integer power

5.6 Singleton Classes

For memory efficiency, many standard-library gates are singletons — one shared Python object per gate type. The singleton variants are:

Standard Class	Singleton Equivalent
Instruction	SingletonInstruction
Gate	SingletonGate
ControlledGate	SingletonControlledGate

Rule: Always call `.toMutable()` before modifying any gate instance. You cannot assume a gate is mutable — it may be a singleton.

6 Special Instructions

Class	Description
Barrier	Marks parts of the circuit that should be optimized independently. Does not execute on hardware.
Delay	Inserts a real-time wait period — used for timing and dynamical decoupling
Measure	Measures a Qubit into a Clbit — irreversible, non-unitary
Reset	Irreversibly resets a qubit to $ 0\rangle$ — non-unitary

Store	Writes a real-time classical expression to a classical storage location
-------	---

7 Control Flow Operations

Control flow operations are subclasses of `ControlFlowOp` and allow circuits to branch, loop, and make real-time decisions during execution.

Class	Behaviour
<code>IfElseOp</code>	Conditional execution — takes two subcircuits (if-body and optional else-body)
<code>SwitchCaseOp</code>	Multi-way conditional — selects one of many subcircuits based on a classical value
<code>ForLoopOp</code>	Loops over a fixed range or set of values — for loop semantics
<code>WhileLoopOp</code>	Repeats a subcircuit until a classical condition is False
<code>BreakLoopOp</code>	Breaks out of the nearest enclosing loop (like <code>break</code> in Python)
<code>ContinueLoopOp</code>	Skips to the next iteration of the enclosing loop (like <code>continue</code> in Python)
<code>BoxOp</code>	Groups instructions together — can carry annotation metadata for compiler passes

```
from qiskit.circuit import QuantumCircuit, QuantumRegister, ClassicalRegister

qr = QuantumRegister(1); cr = ClassicalRegister(1)
circ = QuantumCircuit(qr, cr)
circ.h(0)
circ.measure(0, 0)

# IfElseOp - apply X gate only if measurement result is 1
with circ.if_test((cr, 1)):
    circ.x(0)
```

7.1 Annotation

`BoxOp` can carry metadata via custom subclasses of `Annotation`. These are consumed by compiler passes or backend processing. Defined in `qiskit.circuit.annotation`.

8 Real-Time Classical Computation

Qiskit supports classical expressions evaluated by the QPU controller in real time during a single circuit shot. These live in the `qiskit.circuit.classical` module.

8.1 Core Classes

Class	Role
Var	A typed classical storage location in the circuit — like a variable
Expr	A real-time-evaluated classical expression — the base of all expression nodes
Type	The classical type of an expression — e.g., <code>Bool()</code> , <code>Uint(n)</code>
Value	A constant scalar value node in the expression tree
Unary	A unary operation node — <code>BIT_NOT</code> (bitwise negation) or <code>LOGIC_NOT</code> (logical negation)
Binary	A binary operation node — arithmetic, bitwise, comparison, logical
Cast	A type cast node — explicit or implicit promotion between types

8.2 `expr.lift()` — The Entry Point

`lift()` wraps a plain Python value or circuit object into the appropriate expression tree node:

```
from qiskit.circuit import ClassicalRegister
from qiskit.circuit.classical import expr, types

# Lift a ClassicalRegister -> Var node with inferred Uint type
expr.lift(ClassicalRegister(3, 'c'))
# → Var(ClassicalRegister(3, 'c'), Uint(3))

# Lift an integer with explicit type
expr.lift(5, types.Uint(4))
# → Value(5, Uint(4))

# If you pass an existing Expr, lift() returns it unchanged
```

Rule: `lift()` cannot be used to cast existing `Expr` nodes — use `expr.cast()` for that instead.

8.3 Available Expression Builders

Function	Operation	Returns
<code>expr.bit_not(x)</code>	Bitwise NOT — flips all bits of x	Same type as x (Bool or Uint)
<code>expr.logic_not(x)</code>	Logical NOT — coerces x to Bool, then negates	Bool
<code>expr.bit_and(x, y)</code>	Bitwise AND	Same type as inputs
<code>expr.bit_or(x, y)</code>	Bitwise OR	Same type as inputs
<code>expr.bit_xor(x, y)</code>	Bitwise XOR	Same type as inputs
<code>expr.logic_and(x, y)</code>	Logical AND	Bool
<code>expr.logic_or(x, y)</code>	Logical OR	Bool
<code>expr.equal(x, y)</code>	Equality comparison <code>x == y</code>	Bool
<code>expr.not_equal(x, y)</code>	Inequality <code>x != y</code>	Bool
<code>expr.less(x, y)</code>	Comparison <code>x < y</code>	Bool
<code>expr.lift(val, type=None)</code>	Promote a Python value to an Expr node	Var or Value
<code>expr.cast(x, type)</code>	Explicit type cast	Cast node

```

from qiskit.circuit.classical import expr, types
from qiskit.circuit import ClassicalRegister

c = ClassicalRegister(3, 'c')

# Bitwise NOT on a 3-bit register
expr.bit_not(c)
# Unary(Unary.Op.BIT_NOT, Var(c, Uint(3)), Uint(3))

# Logical NOT — coerces register to Bool first
expr.logic_not(c)
# Unary(Unary.Op.LOGIC_NOT, Cast(Var(c, Uint(3)), Bool(), implicit=True), Bool())

# Explicit cast from Uint(32) to Uint(8) — lossy, must be explicit
v = expr.lift(5, types.Uint(32))
expr.cast(v, types.Uint(8))
# Cast(Value(5, Uint(32)), Uint(8), implicit=False)

```

9 Compile-Time Parametrization

Compile-time parameters are symbolic algebra expressions resolved before circuit execution. They predate real-time classical expressions and are most commonly used to parametrize gate angles.

Class	Role
Parameter	The atomic symbol — a single named compile-time variable (like a sympy Symbol)
ParameterExpression	A symbolic arithmetic expression built from Parameters — supports +, -, *, /, **
ParameterVector	A convenience container for many Parameters with shared naming (e.g., $\theta[0]$, $\theta[1]$, ...)
ParameterVectorElement	An individual Parameter from a ParameterVector

```
from qiskit.circuit import QuantumCircuit, Parameter, ParameterVector
```

```
# Single parameter
theta = Parameter('θ')
circ = QuantumCircuit(1)
circ.rx(theta, 0)          # Parametrized RX gate
circ.assign_parameters({theta: 1.57}) # Bind the value
```

```
# Parameter vector — convenient for many parameters
params = ParameterVector('p', length=3)
circ2 = QuantumCircuit(3)
for i in range(3):
    circ2.ry(params[i], i)
```

Tip: Parameters integrate directly with `QuantumCircuit.assign_parameters()`. Real-time Var/Expr objects are for mid-circuit classical logic; Parameters are for pre-execution symbolic sweeps (e.g., VQE, QAOA).

10 Creating Custom Instructions and Gates

10.1 Custom Instruction via Subclassing

```
from qiskit.circuit import Instruction, QuantumCircuit

class MyInstruction(Instruction):
    def __init__(self):
        super().__init__('my_inst', num_qubits=1, num_clbits=0, params=[])

    def _define(self): # Called lazily to build the decomposition
        qc = QuantumCircuit(1)
        qc.h(0)
        qc.t(0)
        self.definition = qc
```

10.2 Custom Gate via Subclassing

```

from qiskit.circuit import Gate, QuantumCircuit
import numpy as np

class MyGate(Gate):
    def __init__(self, theta):
        super().__init__('my_gate', num_qubits=1, params=[theta])

    def _define(self):
        qc = QuantumCircuit(1)
        qc.rz(self.params[0], 0)
        self.definition = qc

    def to_matrix(self):
        t = float(self.params[0])
        return np.diag([np.exp(-1j * t / 2), np.exp(1j * t / 2)])

```

10.3 Appending Custom Ops to a Circuit

```

circ = QuantumCircuit(2)
circ.append(MyGate(1.57), [0]) # Append to qubit 0
circ.append(MyInstruction(), [1]) # Append to qubit 1
circ.draw('mpl')

```

10.4 EquivalenceLibrary

Registers multiple decompositions of a gate in different basis sets, used by the BasisTranslator transpiler pass:

Class	Description
EquivalenceLibrary	Immutable database of gate-to-circuit decomposition equivalences
SessionEquivalenceLibrary	A mutable instance used by default by the BasisTranslator compiler pass

11 Complete Example — GHZ State

The GHZ (Greenberger–Horne–Zeilinger) state: $|\psi\rangle = (|000\rangle + |111\rangle) / \sqrt{2}$

```

from qiskit import QuantumCircuit

circ = QuantumCircuit(3)

# H on qubit 0 → superposition |0⟩ + |1⟩
circ.h(0)

```

```
# CX (CNOT) on (0→1) → entangles qubits 0 and 1
circ.cx(0, 1)

# CX (CNOT) on (0→2) → extends entanglement to qubit 2 → GHZ state
circ.cx(0, 2)

circ.draw('mpl')
```

12 Random Circuit Utilities

Function	Description
<code>random.random_circuit(num_qubits, depth)</code>	Generates a random circuit of given width and depth
<code>random.random_circuit_from_graph(graph)</code>	Generates a random circuit from a connectivity graph
<code>random.random_clifford_circuit(num_qubits, depth)</code>	Generates a random Clifford circuit — useful for benchmarking

```
from qiskit.circuit.random import random_circuit

circ = random_circuit(num_qubits=4, depth=5, measure=True)
circ.draw('mpl')
```

13 Quick Reference — Class Hierarchy

Class	Inherits from	Unitary?	Runs on hardware?	Key extras
Operation	(object)	Not required	No (abstract only)	Minimal: name, num_qubits, num_clbits
Instruction	Operation	No	Yes	params, definition, condition
Gate	Instruction	Yes	Yes	to_matrix(), control()
ControlledGate	Gate	Yes	Yes	num_ctrl_qubits, ctrl_state
AnnotatedOperation	Operation	Maybe	No (abstract)	modifiers list

14 Common Pitfalls & Exam Tips

- Bit equality: two bits are equal if they are the same object OR same register name/size/index.
- Register vs Bit: circuits track registers but registers add almost no behavioral difference — except `ClassicalRegister` which can be implicitly cast to `int` for conditionals.
- `CircuitInstruction` is a Rust object — mutating it does NOT update the circuit.
- Instruction $\neq$ Gate: Instructions can be non-unitary (Measure, Reset). Gates are always unitary.
- `definition=None` means the instruction is fully opaque — the compiler cannot look inside it.
- Singleton gates: always call `.to_mutable()` before modifying a gate instance.
- Real-time Var/Expr: for mid-circuit classical logic during one shot. Parameter: for pre-execution symbolic sweep across many shots.
- `lift()` wraps Python values into Expr nodes. `cast()` changes the type of an existing Expr. Do not confuse them.
- Logic operations (`logic_not`, `logic_and`) always return `Bool`. Bitwise operations (`bit_not`, `bit_and`) return the same type as the input.
- `AnnotatedOperation` modifiers (`Inverse`, `Control`, `Power`) are applied lazily — useful for building circuits without computing matrices up front.
- abstract circuit $\rightarrow$ physical circuit requires `transpile()`. `ensure_physical()` only updates metadata — it does NOT transpile.
- `ControlFlowOp` subcircuits run inside a single shot — they are real-time, not compile-time.

Qiskit Developer Certification

Revision Notes — Supplement

Bit-Ordering, Matrix Representation & Qiskit Conventions

15 Bit-Ordering & Conventions

Qiskit uses a consistent but non-obvious set of conventions for bit labelling, ordering, and matrix representation. Understanding these is critical — they differ from many textbooks and other quantum computing frameworks.

15.1 The Core Rule — Little Endian

Qiskit uses Little Endian ordering throughout:

- Qubit 0 is the LEAST significant bit (rightmost in strings and bitstrings)
- Qubit $n-1$ is the MOST significant bit (leftmost in strings and bitstrings)
- Each qubit's value contribution is 2^{label} — so qubit 0 contributes $2^0 = 1$, qubit 1 contributes $2^1 = 2$, etc.

Watch out: Most quantum computing textbooks and many other frameworks use Big Endian (qubit 0 leftmost). Qiskit is the opposite. This is a very common source of confusion in the exam and in practice.

15.2 Qubit Labelling in Circuit Diagrams

In `QuantumCircuit.draw()`, qubits are displayed top to bottom in index order:

Position in diagram	Qubit index	Significance
Topmost wire	Qubit 0	Least significant bit
Second wire	Qubit 1	—
...	...	...
Bottommost wire	Qubit $n-1$	Most significant bit

```
from qiskit import QuantumCircuit
qc = QuantumCircuit(2)
qc.x(1)    # Apply X gate to qubit 1 (second wire, bottom)
qc.draw()
# Output:
# q_0: ——— ← topmost, index 0
#
# q_1: [ X ] ← bottommost, index 1
#
```

Tip: Use `qc.draw(reverse_bits=True)` to flip the diagram so qubit 0 appears at the bottom — matching the textbook convention. This only affects the visualization, NOT the circuit itself.

15.3 String Representation of Bitstrings

When a quantum state or measurement result is displayed as a string, the ordering is:

- Leftmost character = qubit n-1 (most significant)
- Rightmost character = qubit 0 (least significant)

```
# Example: qubit 0 is |+>, qubit 1 is |0>
from qiskit.quantum_info import Statevector
sv = Statevector.from_label('0+') # string is read right-to-left!
# '0+' means: qubit 1 = '0', qubit 0 = '+'

sv.probabilities_dict()
# {'00': 0.5, '01': 0.5}
# '01' means qubit 1 = 0, qubit 0 = 1 → decimal value = 0*2 + 1*1 = 1
# '00' means qubit 1 = 0, qubit 0 = 0 → decimal value = 0
```

Bitstring	Qubit 1 (left)	Qubit 0 (right)	Decimal value
'00'	0	0	0
'01'	0	1	1 (= 2^0)
'10'	1	0	2 (= 2^1)
'11'	1	1	3 (= $2^1 + 2^0$)

15.4 Integer Interpretation of Measurement Results

When results are read as integers, bit 0 is the least significant bit:

```
from qiskit import QuantumCircuit
from qiskit.primitives import StatevectorSampler as Sampler

qc = QuantumCircuit(2)
qc.x(1) # Set qubit 1 = 1, qubit 0 stays 0
qc.measure_all()

job = Sampler().run([qc])
result = job.result()
print(result[0].data.meas.get_counts())
# {'10': 1024}
# String '10': qubit 1 = 1 (left), qubit 0 = 0 (right)
# Integer value = 1*2^1 + 0*2^0 = 2
```

15.5 Statevector Matrix (Amplitude) Ordering

When a statevector is stored as a list/array of complex amplitudes, the index directly corresponds to the decimal value of the computational basis state:

Index	Basis state	Qubit 1	Qubit 0
sv[0]	00>	0	0
sv[1]	01>	0	1

sv[2]	10⟩	1	0
sv[3]	11⟩	1	1

```
from qiskit.quantum_info import Statevector
sv = Statevector.from_label('0+') # qubit 1=|0>, qubit 0=|+>

print(sv[1]) # amplitude of |01> - qubit 1=0, qubit 0=1
# (0.7071+0j) ← non-zero because qubit 0 is in |+>

print(sv[2]) # amplitude of |10> - qubit 1=1, qubit 0=0
# 0j ← zero because qubit 1 is in |0>
```

15.6 Gate Matrix Representation Convention

Gate matrices in Qiskit follow from the tensor product convention. For a gate acting on qubit i in an n -qubit system, the full matrix is built using tensor products, with qubit 0 being the rightmost factor:

- Full matrix = $I \otimes I \otimes \dots \otimes \text{Gate_on_qubit_i} \otimes \dots \otimes I$
- Qubit 0 is the rightmost (least significant) factor in the tensor product
- Qubit $n-1$ is the leftmost (most significant) factor

```
# Applying H to qubit 0 in a 2-qubit circuit gives matrix: I ⊗ H
# (NOT H ⊗ I - because qubit 0 is the rightmost factor)
#
# I ⊗ H = [ H  0 ] = [ 1/√2  1/√2  0    0    ]
#           [ 0  H ]   [ 1/√2 -1/√2  0    0    ]
#           [ 0  0 ]   [ 0      0    1/√2  1/√2 ]
#           [ 0  0 ]   [ 0      0    1/√2 -1/√2 ]
```

Common mistake: Many textbooks write 'H on qubit 0' as $H \otimes I$. In Qiskit, it is $I \otimes H$, because qubit 0 is the rightmost (least significant) factor. If you use the wrong order, you get a transposed matrix.

15.7 CX (CNOT) Gate Matrix — The Classic Example

The CX gate with qubit 0 as control and qubit 1 as target is the most-tested convention example:

```
from qiskit import QuantumCircuit
qc = QuantumCircuit(2)
qc.cx(0, 1) # control=qubit 0, target=qubit 1

# This performs: |01> ↔ |11> (flips qubit 1 when qubit 0 is 1)
#
# Matrix (rows/cols indexed by decimal value of |q1 q0>):
#
#           |00> |01> |10> |11>
# |00> [  1    0    0    0  ]
```



```
# |01> [ 0 0 0 1 ]
# |10> [ 0 0 1 0 ]
# |11> [ 0 1 0 0 ]
```

Why this matrix? The CX flips qubit 1 when qubit 0=1. States |01> (q0=1,q1=0) and |11> (q0=1,q1=1) are swapped. The matrix entry at row i, col j is 1 if basis state j maps to basis state i.

15.8 Controlled Gate Conventions

Convention	Rule
Argument order	Controlled gates take (control_qubit, target_qubit) — control always first
ctrl_state	Specifies which control state activates the gate — default is '1' (all controls high)
ctrl_state for CCX	Interpreted as a bitstring of the control qubits using zero-based labelling
Matrix convention	Control qubits come first in the tensor product — matching Qiskit's qubit ordering

```
# CX: control=0, target=1 (default ctrl_state='1')
qc.cx(0, 1)

# CCX (Toffoli): two controls, one target
qc.ccx(control_qubit1=0, control_qubit2=1, target_qubit=2)

# CCX with custom ctrl_state: only fires when control0=1, control1=0
qc.ccx(0, 1, 2, ctrl_state='01')
```

15.9 Full Conventions Summary

Context	Qubit 0 position	Qubit n-1 position
Circuit diagram (default)	Topmost wire	Bottommost wire
Circuit diagram (reverse_bits=True)	Bottommost wire	Topmost wire
Bitstring / measurement string	Rightmost character	Leftmost character
Statevector from_label() string	Rightmost character	Leftmost character
Statevector array index	Least significant bit of index	Most significant bit of index
Integer value of state	$2^0 = 1$ contribution	$2^{(n-1)}$ contribution
Tensor product	Rightmost factor	Leftmost factor
Gate args: cx(ctrl, target)	Can be either — specified explicitly	Can be either — specified explicitly

15.10 How to Reverse the Diagram Without Changing the Circuit

```
# 1. Only changes visualization – circuit is UNCHANGED
qc.draw(reverse_bits=True)

# 2. Returns a NEW circuit with qubit labels reversed
#     (this DOES change which qubit gates act on)
new_qc = qc.reverse_bits()
new_qc.draw()    # qubit 0 is now at the bottom
```

Key distinction: `draw(reverse_bits=True)` only flips the picture. `qc.reverse_bits()` returns a new circuit where the gate-qubit assignments are actually reversed — different circuits!

QISKIT DEVELOPER CERTIFICATION

Transpilation Study Guide

Topics Covered

1. Transpilation Defaults & Configuration Options
2. Setting the Transpiler Optimization Level
3. Commonly Used Parameters for Transpilation
4. Representing Quantum Computers (Backend & Target)
5. Dynamical Decoupling with Pass Managers

Based on IBM Quantum Documentation | Qiskit v2.3+

Topic 1: Transpilation Defaults & Configuration Options

Abstract quantum circuits cannot be run directly on real quantum hardware. A QPU (quantum processing unit) supports only a limited set of basis gates and a fixed qubit connectivity. The transpiler converts your circuit into a form compatible with the target hardware, handling gate translation, qubit mapping, and SWAP insertion.

1.1 The Core Function: `generate_preset_pass_manager`

This is the primary entry point for transpilation in Qiskit. It returns a `StagedPassManager` configured with preset defaults.

```
from qiskit.transpiler import generate_preset_pass_manager
pm = generate_preset_pass_manager(optimization_level=1, backend=backend)
transpiled_circ = pm.run(qc)
```

The function accepts one required argument (`optimization_level`) and many optional ones. All optional arguments have sensible defaults.

1.2 Two Categories of Parameters

A) Parameters Describing the Compilation Target

These tell the transpiler about the hardware it is compiling for:

- `backend` — The full backend object. Sets coupling map, basis gates, and more. Use this OR target, not both.
- `target` — A `Target` object with device constraints. Overrides backend if both specified.
- `backend_properties` — Gate errors, readout errors, coherence times. Retrieved via `backend.properties()`.
- `coupling_map` — Which qubits are physically connected. Can be a `CouplingMap` or adjacency list.
- `basis_gates` — List of gate names the circuit must be translated to (e.g., ['sx', 'rz', 'cx']).
- `timing_constraints` — Hardware restrictions on instruction timing (`granularity`, `min_length`, `pulse_alignment`, `acquire_alignment`).
- `inst_map` — Instruction-to-pulse schedule mapping.

B) Parameters Influencing Transpiler Behaviour

These affect how the transpiler performs each stage:

- `initial_layout` — Fix which physical qubits your virtual qubits map to.
- `layout_method` — Algorithm for the layout stage: 'trivial', 'dense', 'sabre' (default).
- `routing_method` — Algorithm for inserting SWAPs: 'basic', 'lookahead', 'sabre' (default), 'none'.

- `translation_method` — Gate translation strategy: 'translator' (default), 'synthesis', etc.
- `scheduling_method` — 'asap' (as soon as possible) or 'alap' (as late as possible, default).
- `approximation_degree` — Float 0.0–1.0 controlling synthesis accuracy (1.0 = no approximation).
- `seed_transpiler` — Integer seed for reproducibility in stochastic passes.
- `unitary_synthesis_method` — Synthesis method for unitary gates; default is 'default'.
- `hls_config` — Configuration for the HighLevelSynthesis pass.

1.3 Full Default Configuration

If you call `generate_preset_pass_manager` without any optional parameters, these are the defaults used:

```
generate_preset_pass_manager(
    optimization_level=1,
    backend=None,          target=None,
    basis_gates=None,      coupling_map=None,
    initial_layout=None,   layout_method=None,
    routing_method=None,   translation_method=None,
    scheduling_method=None,
    approximation_degree=1.0,
    seed_transpiler=None,
    unitary_synthesis_method='default',
    unitary_synthesis_plugin_config=None,
    hls_config=None,
    init_method=None,
    optimization_method=None,
)
```

□ **EXAM TIP:** Any parameter you explicitly set overrides the transpiler's automatic choices for that aspect. For example, setting `initial_layout` pins qubit mapping and prevents the transpiler from choosing different qubits.

□ **NOTE:** If you specify both `backend` and `coupling_map`, the `coupling_map` overrides the backend's connectivity information for that parameter only.

Topic 2: Setting the Transpiler Optimization Level

The `optimization_level` argument controls how aggressively the transpiler optimizes your circuit. It is an integer from 0 to 3. Higher levels produce shorter, more hardware-efficient circuits, but take longer to compile. Since two-qubit gates (like ECR or CX) are the primary source of hardware errors, reducing their count is the main optimization goal.

2.1 Optimization Level Comparison Table

Level	What It Does	Typical 2Q Gates
Level 0	No optimization. Trivial layout (virtual $\rightarrow$ physical qubit 1:1). SWAPs inserted via SabreSwap. Used for hardware characterization.	6 ECR gates
Level 1	Light optimization. TrivialLayout attempted first, then SabreSwap + VF2LayoutPostLayout. InverseCancellation for 1Q gates. Default.	6 ECR gates
Level 2	Medium optimization. Same as Level 1 (without trivial) + CommutativeCancellation. Deeper search for layout/routing.	6 ECR gates
Level 3	High optimization. Resynthesis of 2Q blocks via KAK Decomposition. OptimizeSwapBeforeMeasure, RemoveDiagonalGatesBeforeMeasure.	3 ECR gates

2.2 Layout Algorithms by Level

A key difference between levels is which layout algorithm is used:

- Level 0: TrivialLayout only — virtual qubit N maps to physical qubit N. Simple but often suboptimal.
- Level 1: TrivialLayout first, then falls back to SabreSwap + VF2LayoutPostLayout if SWAPs are needed.
- Level 2: Skips TrivialLayout entirely. Uses heuristic optimization with deeper search.
- Level 3: Same as Level 2 with greater effort, plus two-qubit gate resynthesis via KAK Decomposition.

2.3 Level 3 and KAK Decomposition

At Level 3, the transpiler applies Cartan's KAK Decomposition to re-synthesize two-qubit gate blocks from scratch. This is powerful because any two-qubit unitary can always be implemented using at most three CX (or ECR) gates. Re-synthesis can therefore eliminate redundant gates that accumulated during translation.

Example: A random unitary + SWAP gate compiles to:

- Level 0 / 1 / 2: 6 ECR gates

- Level 3 (exact): 3 ECR gates
- Level 3 + approximation (degree=0.99): 2 ECR gates (with ~0.8% fidelity loss)

2.4 Combining with approximation_degree

At Level 3, you can allow the transpiler to make approximations to reduce the gate count further:

```
pm = generate_preset_pass_manager(
    optimization_level=3,
    approximation_degree=0.99,    # 1% approximation allowed
    backend=backend,
    seed_transpiler=12345,
)
```

Setting `approximation_degree < 1.0` tells the transpiler it may produce a circuit whose unitary is not exactly equal to the input, but is within the specified fidelity. This can reduce expensive two-qubit gates, which may ultimately improve results on noisy hardware by reducing hardware error even as synthesis error increases.

□ **EXAM TIP:** Increasing the optimization level does not always improve results. Levels 1 and 2 often produce the same circuit. Level 3 is most beneficial for circuits with large two-qubit gate blocks.

□ **NOTE:** Unitarity-breaking passes at Level 3 (`OptimizeSwapBeforeMeasure`, `RemoveDiagonalGatesBeforeMeasure`) rearrange or remove gates relative to measurements. Use with care if you need the exact unitary.

Topic 3: Commonly Used Parameters for Transpilation

This topic covers the most frequently configured parameters in `generate_preset_pass_manager` and `transpile()`, grouped by function.

3.1 approximation_degree

Controls how closely the output circuit must match the input circuit's unitary, as a float from 0.0 (maximum approximation) to 1.0 (no approximation, the default).

How it works: In two-qubit unitary synthesis (used in the initialization stage at all levels, and in the optimization stage at Level 3), this value sets the target fidelity. Lower values mean the output gate sequence can deviate more from the ideal unitary — but the benefit is fewer gates.

```
# Example: 2 CX gates instead of 3, at cost of slight fidelity loss
pm = generate_preset_pass_manager(
    optimization_level=1,
    approximation_degree=0.85,
    basis_gates=['sx', 'rz', 'cx'],
)
```

A value of 0.85 might yield 2 CX gates where the exact decomposition needs 3. This technique was used to increase Quantum Volume on IBM devices.

❏ **EXAM TIP:** For noisy devices, a slightly approximate circuit with fewer 2Q gates can outperform an exact circuit with more gates, because hardware gate errors dominate over synthesis approximation errors.

3.2 seed_transpiler

Several transpiler passes are stochastic (they use random choices internally, particularly in layout and routing). This means two runs on the same circuit can produce different results. To get reproducible transpilation, fix the seed:

```
pm = generate_preset_pass_manager(
    optimization_level=1,
    seed_transpiler=42,
)
```

❏ **NOTE:** Always set `seed_transpiler` when comparing results between different transpiler configurations, or when you need consistent benchmarks.

3.3 initial_layout

Maps your circuit's virtual qubits to specific physical qubits on the backend. Before transpilation, qubits in your circuit are 'virtual' — they don't correspond to real hardware qubits. The layout stage decides this mapping.

You can specify `initial_layout` as a `Layout` object or a simple list of integers:

```
from qiskit.transpiler import Layout

# Map virtual qubit a -> physical qubit 5, b -> physical qubit 6
initial_layout = Layout({a: 5, b: 6})

# Or equivalently as a list (index = virtual qubit, value = physical qubit)
initial_layout = [5, 6]
```

Important: The final layout is not guaranteed to match the initial layout exactly, because the transpiler may permute qubits via SWAP gates or other means. The `initial_layout` is a starting suggestion.

❏ **EXAM TIP:** Use `plot_error_map(backend)` to visualize qubit error rates and connectivity before selecting an `initial_layout`. Choose qubits with low gate error on the paths your circuit needs.

3.4 Transpiler Stage Method Options

These `_method` parameters let you select specific algorithms for each transpiler stage:

Parameter	Type / Values	Purpose
<code>layout_method</code>	'trivial', 'dense', 'sabre'	Algorithm for assigning virtual → physical qubits
<code>routing_method</code>	'basic', 'lookahead', 'sabre', 'none'	Algorithm for inserting SWAP gates to satisfy connectivity
<code>translation_method</code>	'translator', 'synthesis', 'unroller'	How gates are decomposed into basis gates
<code>scheduling_method</code>	'asap', 'alap'	When to schedule instructions relative to circuit end
<code>optimization_method</code>	plugin name	Custom optimization stage plugin
<code>init_method</code>	plugin name	Custom initialization stage plugin
<code>unitary_synthesis_method</code>	'default' or plugin	Method for decomposing unitary matrices

To list all installed plugins for a given stage:

```
from qiskit.transpiler import list_stage_plugins
```

```
list_stage_plugins('routing') # shows all routing plugins
```

□ **NOTE:** `scheduling_method='alap'` (default) keeps qubits in the ground state as long as possible, which reduces decoherence. Use `'asap'` only when you have specific timing requirements.

Topic 4: Representing Quantum Computers (Backend & Target)

To transpile a circuit for real hardware, the transpiler needs to know the hardware's constraints. This information is encapsulated in the Backend and Target objects. Understanding how these work is essential for writing efficient, hardware-aware quantum programs.

4.1 Backend vs. Target

Parameter	Type / Values	Purpose
Backend	<code>BackendV2</code> object	Full interface to QPU: submits jobs, holds Target, provides instruction schedule maps
Target	<code>Target</code> object	Hardware constraints: connectivity, basis gates, error rates, timing. Used internally by transpiler

```
from qiskit_ibm_runtime.fake_provider import FakeSherbrooke
backend = FakeSherbrooke()
target = backend.target # extract the Target
```

In most cases, just passing backend to the transpiler is sufficient — it will extract the Target automatically.

4.2 The Coupling Map

The coupling map is a directed graph describing which pairs of physical qubits can participate in two-qubit gates. It determines the 'connectivity' of the device.

- Virtual qubits are qubits in your abstract circuit (labelled by index).
- Physical qubits are real hardware qubits (also labelled by integers).
- The layout stage maps virtual → physical qubits.
- The routing stage inserts SWAP gates to move quantum information between non-adjacent qubits.

```
coupling_map = target.build_coupling_map()
pm = generate_preset_pass_manager(
    optimization_level=1,
    coupling_map=coupling_map,
    seed_transpiler=12345,
)
```

At Level 0 with TrivialLayout, virtual qubit N maps to physical qubit N linearly. This often requires many SWAP gates because the circuit's connectivity rarely matches the device's. At

Level 1+, VF2LayoutPostLayout finds a ring or subgraph of physical qubits that matches the circuit's connectivity, eliminating SWAPs entirely in many cases.

□ **EXAM TIP:** Visualize the selected qubit layout using: `plot_circuit_layout(transpiled_circuit, backend, view='physical')`. This shows exactly which physical qubits were chosen.

4.3 Basis Gates (Supported Instruction Set)

Every QPU supports only a specific set of gates — its 'native gate set'. All gates in your circuit must be translated to this set. The BasisTranslator pass performs this translation.

```
basis_gates = list(target.operation_names)
# Example output for FakeSherbrooke:
# ['rz', 'ecr', 'reset', 'measure', 'id', 'x', 'sx', 'delay', ...]
```

FakeSherbrooke native gates: single-qubit gates are rz, x, sx; the two-qubit gate is ecr (echoed cross-resonance). Note that CX gates are not native — they are constructed from ecr. Different QPUs may use cx as the two-qubit basis gate.

□ **NOTE:** The ECR gate is the entangling part of the CX gate. On IBM backends that use ECR as the native 2Q gate, CX gates decompose to ECR + single-qubit gates.

4.4 Device Error Rates

The Target can contain per-gate, per-qubit error rates and durations. This information allows the transpiler to choose the best physical qubits — preferring those with lower error rates.

```
# Query error rate of ECR gate between qubits 1 and 0
target['ecr'][(1, 0)]
# InstructionProperties(duration=5.3e-7, error=0.0075)
```

To provide error information to the transpiler without the full backend object, you can build a custom Target using `Target.from_configuration()` and populate it with `InstructionProperties` objects. When error info is present, the VF2PostLayout pass actively selects qubits with lower error rates.

Summary: What information does the transpiler need?

- Minimum required: `coupling_map` — to know qubit connectivity.
- Better: `coupling_map` + `basis_gates` — to translate gates correctly.
- Best: full Target or Backend — includes error rates for optimal qubit selection.

□ **EXAM TIP:** Passing only `coupling_map` without `basis_gates` means the transpiler won't know which gates to translate to. Always pair them, or just use the backend/target directly.

Topic 5: Dynamical Decoupling with Pass Managers

Dynamical Decoupling (DD) is an error suppression technique applied after a circuit has been transpiled and scheduled. It targets idle qubits — qubits that are waiting while other qubits are active — by inserting carefully designed pulse sequences that cancel out environmental noise through symmetry.

5.1 What is Dynamical Decoupling?

When a qubit is idle (not being operated on), it still interacts with its environment and accumulates phase errors. These errors are called decoherence. Dynamical decoupling works by inserting a sequence of pulses into the idle window that:

- Form an identity operation (they cancel each other out, preserving the qubit state).
- Are equally spaced in time throughout the idle period.
- Flip the qubit around the Bloch sphere in a pattern that causes noise effects to cancel.

The analogy is noise-cancelling headphones: the DD pulses generate the 'negative' of the noise to make it disappear. This technique was adapted from NMR (nuclear magnetic resonance) spin refocusing.

□ **EXAM TIP:** Dynamical decoupling is most effective in circuits with sequential two-qubit gates (like ansatz circuits), where many qubits sit idle while others are active. The `efficient_su2` ansatz is a perfect example.

5.2 The DD Workflow: Three Required Passes

Because DD modifies the timing of an already-compiled circuit, it must be applied after standard transpilation. Three passes are involved:

Parameter	Type / Values	Purpose
<code>ALAPScheduleAnalysis</code>	<code>Scheduling pass</code>	Analyzes timing: records when each operation starts/ends and when idle windows exist. Must run first.
<code>PadDynamicalDecoupling</code>	<code>DD insertion pass</code>	Inserts the DD sequence into idle windows at regular intervals.
<code>BasisTranslator</code>	<code>Translation pass</code>	Converts any non-native gates (e.g. <code>YGate</code>) back into the backend's basis gates after DD insertion.

5.3 The XY4 Sequence

The guide uses the XY4 sequence — four gates [X, Y, X, Y] — as the DD sequence. This is a well-known decoupling sequence that suppresses both X-axis and Y-axis noise to first order.

```
from qiskit.circuit.library import XGate, YGate
dd_sequence = [XGate(), YGate(), XGate(), YGate()]
```

The X gate is a native gate on IBM backends. The Y gate is NOT a native gate, so its timing information must be added to the Target manually before the scheduling pass can work with it.

5.4 Step-by-Step Implementation

Step 1: Set up backend and transpile the circuit first

```
from qiskit_ibm_runtime.fake_provider import FakeSherbrooke
from qiskit.transpiler import generate_preset_pass_manager
from qiskit.circuit.library import efficient_su2

backend = FakeSherbrooke()
target = backend.target
basis_gates = list(target.operation_names) # save BEFORE modifying target

qc = efficient_su2(12, entanglement='circular', reps=1)
pm = generate_preset_pass_manager(1, target=target, seed_transpiler=12345)
qc_t = pm.run(qc)
```

Step 2: Add YGate timing to the Target

Since YGate is not a native gate, you must inform the target of its duration and error (copied from XGate, since they are physically identical in terms of implementation):

```
from qiskit.transpiler import InstructionProperties
from qiskit.circuit.library import YGate

y_props = {}
for qubit in range(target.num_qubits):
    y_props[(qubit,)] = InstructionProperties(
        duration=target['x'][(qubit,)].duration,
        error=target['x'][(qubit,)].error,
    )
target.add_instruction(YGate(), y_props)
```

Step 3: Bind parameters (for parameterized circuits)

```
import numpy as np
rng = np.random.default_rng(1234)
qc_t.assign_parameters(rng.uniform(-np.pi, np.pi, qc_t.num_parameters),
    inplace=True)
```

Step 4: Build and run the DD PassManager

```
from qiskit.transpiler import PassManager
from qiskit.transpiler.passes.scheduling import (
    ALAPScheduleAnalysis, PadDynamicalDecoupling
)

dd_pm = PassManager([
    ALAPScheduleAnalysis(target=target),
    PadDynamicalDecoupling(target=target, dd_sequence=dd_sequence),
])
qc_dd = dd_pm.run(qc_t)
```

Step 5: Re-apply BasisTranslator

YGate is not a native gate, so the circuit must be translated back to basis gates. The default pipeline ran BasisTranslator before scheduling, so it must be reapplied manually:

```
from qiskit.circuit.equivalence_library import SessionEquivalenceLibrary as sel
from qiskit.transpiler.passes import BasisTranslator

qc_dd = BasisTranslator(sel, basis_gates)(qc_dd)
```

The final circuit will contain explicit Delay gates (representing idle time) and the DD pulse sequences woven through those delays. It is now ready to submit to the backend.

5.5 Why ALAP Scheduling?

ALAP (As Late As Possible) scheduling delays the start of each gate as long as possible, which keeps qubits in the ground state ($|0\rangle$) for longer. This reduces decoherence since ground-state qubits are more stable. ALAP is also the scheduling mode required by PadDynamicalDecoupling — it needs fully scheduled circuits with known idle windows before it can insert pulses.

❏ **EXAM TIP:** Use `timeline_drawer(qc_dd, idle_wires=False, target=target)` to visualize the scheduled circuit and confirm that DD sequences appear at regular intervals in idle windows.

❏ **NOTE:** This workflow must always happen in this order: (1) Transpile normally, (2) Bind parameters, (3) Schedule (ALAP), (4) Pad DD, (5) Re-translate to basis gates.

Quick Reference: Key Facts for the Exam

Essential API Calls

```
generate_preset_pass_manager(optimization_level, backend=backend)
pm.run(circuit)                # transpile one or more circuits
backend.target                 # get Target from backend
target.build_coupling_map()    # get CouplingMap from Target
target['gate'][(q,)]           # InstructionProperties: duration, error
target.add_instruction(gate, props_dict)
list_stage_plugins('stage_name')
plot_circuit_layout(qc, backend, view='physical')
timeline_drawer(qc, target=target)
```

Critical Facts to Remember

- optimization_level is the only REQUIRED argument to generate_preset_pass_manager.
- Specify backend OR target, never both (target overrides backend if you do).
- initial_layout maps virtual → physical but final layout may differ due to routing.
- DD must be applied AFTER transpilation and BEFORE submission.
- ALAPScheduleAnalysis must run BEFORE PadDynamicalDecoupling.
- BasisTranslator must be re-run manually AFTER DD if non-native gates were used.
- YGate is not a native gate on FakeSherbrooke — XGate is. Both have the same duration/error.
- ECR gate is the native 2-qubit gate on IBM backends (not CX).
- Any 2Q unitary can be decomposed with at most 3 CX/ECR gates (KAK theorem).
- seed_transpiler ensures reproducibility across stochastic transpiler passes.
- approximation_degree=0.0 is maximum approximation; 1.0 is exact (default).
- VF2PostLayout selects optimal qubits only when error rates are in the Target.

Optimization Level Quick Comparison

- Level 0: TrivialLayout only, no real optimization. Hardware characterization use.
- Level 1: Best balance of speed and quality. Default. Uses VF2 if SWAPs needed.
- Level 2: Slightly deeper search, CommutativeCancellation. Often same as Level 1.
- Level 3: Resynthesis with KAK. Best for circuits with large 2Q gate blocks.

Transpiler Stages Summary

Parameter	Type / Values	Purpose
-----------	---------------	---------

Initialization	<code>init_method</code>	High-level synthesis (HLS), initial circuit decomposition
Layout	<code>layout_method</code>	Maps virtual → physical qubits (trivial, sabre, dense)
Routing	<code>routing_method</code>	Inserts SWAP gates for connectivity (sabre, basic, lookahead)
Translation	<code>translation_method</code>	Translates to basis gates (BasisTranslator)
Optimization	<code>optimization_method</code>	Cancels redundant gates (InverseCancellation, CommutativeCancellation, KAK)
Scheduling	<code>scheduling_method</code>	Assigns timing to gates (asap, alap)

Dynamical Decoupling Checklist

- 1. Transpile the circuit to ISA using `generate_preset_pass_manager`
- 2. Save `basis_gates` BEFORE modifying target
- 3. Add non-native gate properties to target (`YGate` → copy from `XGate`)
- 4. Bind all parameters (parameterized circuits only)
- 5. Run `PassManager([ALAPScheduleAnalysis(target), PadDynamicalDecoupling(target, dd_sequence)])`
- 6. Run `BasisTranslator(SessionEquivalenceLibrary, basis_gates)(qc_dd)`
- 7. Submit the final circuit

❏ **REMEMBER:** The XY4 sequence $[X, Y, X, Y]$ composes to identity and suppresses both X and Y noise channels. The gates must be evenly spaced in time within the idle window.